

Reflection in Go

Inspecting and manipulating types at runtime via the `reflect` package. Powerful, slow, and easy to misuse. Here's how it works, what it's for, and when to pick something else.

What reflection is

Reflection lets your code examine a value's type and structure at runtime, instead of at compile time. You can:

- Discover a value's type and kind.
- Read and modify struct fields by name.
- Call methods or functions by name.
- Build new values of types you didn't know about when you wrote the code.

The cost: type safety is lost (errors surface at runtime), performance drops 10-100x, and the code reads worse than the same thing written with concrete types or generics.

The rule: use it when you genuinely can't know the type at compile time. JSON decoding, template rendering, ORM scanning, validation libraries are the canonical cases. For your own business logic, almost never.

The two central types

```
import "reflect"

reflect.TypeOf(v)           // returns reflect.Type
reflect.ValueOf(v)          // returns reflect.Value
```

`reflect.Type` describes the type. `reflect.Value` carries the actual value (and its type).

```
var x int = 42
t := reflect.TypeOf(x)    // int
v := reflect.ValueOf(x)   // 42

t.Name()                  // "int"
t.Kind()                  // reflect.Int
v.Int()                   // 42 (int64)
v.Interface()             // 42 (as any)
```

Kind vs Type

`Type` is the named type (`Celsius`, `User`, `int`). `Kind` is the underlying kind (`reflect.Float64`, `reflect.Struct`, `reflect.Int`).

```
type Celsius float64
var c Celsius = 100

t := reflect.TypeOf(c)
t.Name()           // "Celsius"
t.Kind()           // reflect.Float64
```

The full list of kinds:

```
Invalid, Bool, Int, Int8, Int16, Int32, Int64,
Uint, Uint8, Uint16, Uint32, Uint64, Uintptr,
Float32, Float64, Complex64, Complex128,
Array, Chan, Func, Interface, Map, Pointer,
Slice, String, Struct, UnsafePointer
```

Switch on `Kind()` when you care about the shape, regardless of the named type. Switch on `Type()` when the named type matters.

Common Value methods

```
v := reflect.ValueOf(x)

v.Kind()           // the kind
v.Type()           // the type
v.IsValid()        // false for zero Value
v.IsNil()          // for pointers, interfaces, slices, maps, chans, funcs
v.IsZero()         // true if x is the zero value of its type

v.Int()            // int64; panics if kind isn't an int kind
v.Uint()           // uint64
v.Float()          // float64
v.Bool()           // bool
v.String()         // string (or string description if not a string)
v.Bytes()          // []byte (only for []byte)
v.Complex()        // complex128
v.Interface()      // any; the original value
v.Pointer()        // uintptr of the underlying pointer (for pointers, slices,
                    // maps, etc.)
```

If you call the wrong accessor (e.g., `v.Int()` on a string value), `reflect` panics. Always check `Kind()` first.

Inspecting structs

```
type User struct {
    ID    int    `json:"id"`
    Name  string `json:"name" validate:"required"`
    Email string `json:"email,omitempty"`
}

u := User{ID: 1, Name: "alice", Email: "a@x.com"}
t := reflect.TypeOf(u)
v := reflect.ValueOf(u)

t.NumField()           // 3
for i := 0; i < t.NumField(); i++ {
    field := t.Field(i)           // reflect.StructField
    value := v.Field(i)           // reflect.Value
    fmt.Println(field.Name, field.Type, field.Tag.Get("json"), value)
}
```

`reflect.StructField` carries the field name, type, tag, anonymous flag, and index path.

Tags

Tags are raw strings on struct fields. `StructTag.Get(key)` parses them.

```
type T struct {
    F string `json:"foo" validate:"required,min=3"`
}

f, _ := reflect.TypeOf(T{}).FieldByName("F")
f.Tag.Get("json")           // "foo"
f.Tag.Get("validate")       // "required,min=3"
f.Tag.Lookup("missing")     // "", false
```

Tags are convention. The format inside is up to each library; `Get` just returns the raw value for the key.

Setting values: addressability

Reading is easy. Writing is restricted: `reflect.Value` can only be set if it's *addressable*. A copy of a value (which is what `reflect.ValueOf(x)` returns for non-pointer `x`) is not addressable.

```

x := 10
v := reflect.ValueOf(x)
v.SetInt(20)                                // panic: not addressable

p := reflect.ValueOf(&x)                    // pointer to x
v = p.Elem()                                // dereference; now addressable
v.SetInt(20)
fmt.Println(x)                              // 20

```

The rule: to modify a value with reflection, you must start with a pointer and call `Elem()` to get the addressable value behind it.

For structs:

```

u := User{}
v := reflect.ValueOf(&u).Elem()
v.FieldByName("Name").SetString("alice")

```

Same for slices and maps:

```

s := []int{1, 2, 3}
v := reflect.ValueOf(s)
v.Index(0).SetInt(99)                       // works: slice elements are addressable through
      the slice header                        // [99 2 3]
fmt.Println(s)

m := map[string]int{"a": 1}
v = reflect.ValueOf(m)
v.SetMapIndex(reflect.ValueOf("b"), reflect.ValueOf(2)) // map insert
// map values are NOT addressable; you can't get a Value and SetInt on it

```

Exported vs unexported fields

You can only `Set` exported fields. Unexported fields are readable but not writable through reflection.

```

type T struct {
    Public string
    private string                // can read with .Field(1), but .Set... panics
}

```

There are tricks with `unsafe` to bypass this. Don't.

Creating values

```
t := reflect.TypeOf(User{})
v := reflect.New(t)           // *User, points to a zeroed User
elem := v.Elem()              // the User value
elem.FieldByName("Name").SetString("alice")

asUser := v.Interface().(*User) // convert back to typed value
fmt.Println(asUser)             // &{0 alice }
```

`reflect.New(t)` is the runtime equivalent of `new(T)`. It returns a `reflect.Value` holding a pointer to a freshly zeroed `T`.

For slices:

```
t := reflect.TypeOf([]int{})
slc := reflect.MakeSlice(t, 3, 5) // len 3, cap 5
slc.Index(0).SetInt(10)
```

For maps:

```
mt := reflect.MapOf(reflect.TypeOf(""), reflect.TypeOf(0)) // map[string]int type
m := reflect.MakeMap(mt)
m.SetMapIndex(reflect.ValueOf("a"), reflect.ValueOf(1))
```

For channels:

```
ct := reflect.ChanOf(reflect.BothDir, reflect.TypeOf(0))
ch := reflect.MakeChan(ct, 10)
ch.Send(reflect.ValueOf(42))
v, ok := ch.Recv()
```

Calling functions and methods

```
type Greeter struct{ Name string }
func (g Greeter) Greet(prefix string) string {
    return prefix + " " + g.Name
}

g := Greeter{Name: "alice"}
v := reflect.ValueOf(g)

m := v.MethodByName("Greet")
results := m.Call([]reflect.Value{reflect.ValueOf("hi")})
fmt.Println(results[0].String())    // "hi alice"
```

`Call` takes a slice of `reflect.Value` (one per argument) and returns a slice of `reflect.Value` (one per return). Argument types must match exactly.

For variadic functions, use `CallSlice` and pass the variadic args as a single slice value.

Function values

```
add := func(a, b int) int { return a + b }
v := reflect.ValueOf(add)
results := v.Call([]reflect.Value{
    reflect.ValueOf(2),
    reflect.ValueOf(3),
})
fmt.Println(results[0].Int())    // 5
```

Inspecting interfaces

When you call `reflect.ValueOf(interfaceValue)`, the reflection sees the *dynamic* type, not the interface type.

```
var r io.Reader = strings.NewReader("hi")
t := reflect.TypeOf(r)           // *strings.Reader, NOT io.Reader
```

To get the static interface type, you need a pointer to an interface variable:

```
ti := reflect.TypeOf((*io.Reader)(nil)).Elem()
fmt.Println(ti)           // io.Reader
fmt.Println(ti.Kind())    // reflect.Interface
```

The `(*T)(nil)` trick is the idiomatic way to get a type without an actual value.

Implements check

```
errType := reflect.TypeOf((*error)(nil)).Elem()
t := reflect.TypeOf(myValue)
if t.Implements(errType) {
    // myValue's type implements error
}
```

Comparing values

`reflect.DeepEqual` does a recursive comparison. Used in `testify` and many other libraries.

```
reflect.DeepEqual([]int{1, 2}, []int{1, 2})           // true
reflect.DeepEqual(map[string]int{"a": 1}, map[string]int{"a": 1}) // true
reflect.DeepEqual(User{1, "a"}, User{1, "a"})         // true
```

Rules: - Slices: same length and each element equal. - Maps: same keys and each value equal. - Structs: each exported and unexported field equal. - Pointers: pointed-to values equal, or both nil. - Functions: only equal if both nil.

Faster alternatives exist (`google/go-cmp` with options, custom `Equal` methods). For tests, `cmp.Diff` is often clearer because it shows what's different.

Real-world use cases

JSON encoding

`encoding/json` uses reflection to walk an arbitrary struct, read field names and tags, and produce the JSON.

```
type User struct {
    Name string `json:"name"`
    Age  int    `json:"age,omitempty"`
}

json.Marshal(User{Name: "alice"}) // {"name":"alice"}
```

The `Marshal` function takes any, reflects on the type at runtime, and walks the struct field-by-field. This is why JSON is slower than hand-rolled codecs.

For high-throughput JSON, libraries like `easyjson` and `go-json` generate type-specific code that avoids reflection.

Template rendering

`text/template` and `html/template` look up fields and methods on the data object by name. Reflection is how `.User.Name` in a template resolves to `data.User.Name` at runtime.

Database row scanning

`database/sql.Rows.Scan(&v1, &v2, ...)` uses reflection to write into the pointers based on the column types. ORMs (GORM, sqlx, bun) push reflection further to map full structs.

Validation libraries

`go-playground/validator` reads `validate:"..."` tags via reflection, runs the rules on each field, and reports errors.

```
type User struct {  
    Email string `validate:"required,email"`  
    Age   int    `validate:"gte=18"`  
}
```

Dependency injection containers

`google/wire` generates code at build time (no reflection). `uber-go/fx` uses reflection to resolve dependencies at runtime.

Performance reality

Approximate cost compared to direct access (Go 1.21, 64-bit):

Operation	Cost
Direct struct field read	<1 ns
<code>reflect.ValueOf(v)</code>	~10-20 ns
<code>v.Field(i).Int()</code>	~10-30 ns
<code>v.FieldByName("X")</code>	~100-300 ns (string lookup)
<code>v.MethodByName("X").Call(args)</code>	~500-2000 ns
<code>reflect.DeepEqual</code> on small struct	~200-500 ns
JSON encode/decode	dominated by reflection + allocation

The 100x penalty isn't theoretical. A JSON-heavy service spends meaningful percentages of CPU inside `reflect`.

Caching reflection

If you must use reflection in a hot path, cache the type and field lookups:

```
type Encoder struct {
    fields []reflect.StructField
    tags   []string
}

var encoderCache sync.Map           // map[reflect.Type]*Encoder

func encoderFor(t reflect.Type) *Encoder {
    if e, ok := encoderCache.Load(t); ok { return e.(*Encoder) }
    e := buildEncoder(t)
    encoderCache.Store(t, e)
    return e
}
```

Build the encoder once per type. Reuse for every value. This is how the JSON package handles repeated calls.

Alternatives to reflection

Generics (Go 1.18+)

For functions that need to work over many types at compile time, generics beat reflection on every axis: faster, type-safe, readable.

```
// reflection
func MaxAny(s []any) any { ... } // unsafe, slow

// generics
func Max[T cmp.Ordered](s []T) T { ... }
```

If the type set is bounded at compile time, use generics.

Code generation

Tools like `stringer`, `easyjson`, `protoc-gen-go` read source and emit specialized code. Run-time cost is zero. Build-time cost is real.

```
//go:generate stringer -type=Status
```

Use codegen for repetitive code on known types: enum stringers, marshalers, mocks.

Interfaces

When you need polymorphism, an interface is faster and clearer than reflection-based “does this type have method X.”

```
// reflection
m := reflect.ValueOf(v).MethodByName("Close")
if m.IsValid() { m.Call(nil) }

// interface
if c, ok := v.(io.Closer); ok { c.Close() }
```

Use the second always. Reflection here adds runtime cost and removes compile-time safety with nothing to show for it.

Type switches

```
switch v := any.(type) {
case int:    handleInt(v)
case string: handleString(v)
}
```

Much faster than `reflect.TypeOf(any).Kind()` switching. Type switches are compile-time-checked.

When to actually reach for reflect

Use cases where reflection is the right tool:

1. **Generic serialization** to/from a format that's defined by the value's shape (JSON, XML, msgpack, protobuf via wire format).
2. **Library code that processes user-defined types** the library author can't know in advance (template engines, validators, ORM mappers).
3. **Runtime field/method introspection** for diagnostics, debugging, REPL-style tools.
4. **Test helpers** comparing arbitrary structures (`reflect.DeepEqual` , `go-cmp`).

Reasons that look right but probably aren't:

- “I want to write a function that handles any type.” Use generics.
 - “I want to check if a value has a method.” Use an interface assertion.
 - “I want to dispatch on type at runtime.” Use a type switch.
 - “I want to copy fields between structs of different types.” Use codegen or write a typed converter.
-

Worked example: generic deep copy

A small reflection-based deep copier:

```
func DeepCopy[T any](v T) T {
    src := reflect.ValueOf(v)
    dst := reflect.New(src.Type()).Elem()
    copyValue(dst, src)
    return dst.Interface().(T)
}

func copyValue(dst, src reflect.Value) {
    switch src.Kind() {
    case reflect.Struct:
        for i := 0; i < src.NumField(); i++ {
            copyValue(dst.Field(i), src.Field(i))
        }
    case reflect.Slice:
        if src.IsNil() { return }
        dst.Set(reflect.MakeSlice(src.Type(), src.Len(), src.Cap()))
        for i := 0; i < src.Len(); i++ {
            copyValue(dst.Index(i), src.Index(i))
        }
    case reflect.Map:
        if src.IsNil() { return }
        dst.Set(reflect.MakeMapWithSize(src.Type(), src.Len()))
        iter := src.MapRange()
        for iter.Next() {
            newVal := reflect.New(src.Type().Elem()).Elem()
            copyValue(newVal, iter.Value())
            dst.SetMapIndex(iter.Key(), newVal)
        }
    case reflect.Pointer:
        if src.IsNil() { return }
        dst.Set(reflect.New(src.Type().Elem()))
        copyValue(dst.Elem(), src.Elem())
    default:
        dst.Set(src)
    }
}
```

This handles structs, slices, maps, and pointers recursively. Real production libraries (like [jinzhu/copier](#)) extend this with cycle detection, custom hooks, and field name matching across types.

Common patterns

Field walker

```
func WalkFields(v any, fn func(name string, val reflect.Value)) {
    rv := reflect.Indirect(reflect.ValueOf(v))
    rt := rv.Type()
    for i := 0; i < rt.NumField(); i++ {
        fn(rt.Field(i).Name, rv.Field(i))
    }
}
```

Useful for validation, logging redaction, audit trails.

Tag-driven configuration

```
type Config struct {
    Port    int    `env:"PORT" default:"8080"`
    DBHost  string `env:"DB_HOST"`
}

func LoadFromEnv(cfg any) error {
    rv := reflect.ValueOf(cfg).Elem()
    rt := rv.Type()
    for i := 0; i < rt.NumField(); i++ {
        field := rt.Field(i)
        envKey := field.Tag.Get("env")
        if envKey == "" { continue }
        val := os.Getenv(envKey)
        if val == "" {
            val = field.Tag.Get("default")
        }
        setField(rv.Field(i), val)
    }
    return nil
}
```

This is roughly how `envconfig` works.

Interface check by type

```
func IsErrorLike(v any) bool {
    t := reflect.TypeOf(v)
    return t.Implements(reflect.TypeOf((*error)(nil)).Elem())
}
```

(In practice, use a type assertion instead.)

Gotchas with reflection

Panic-driven API

Many `reflect` methods panic on invalid use. Defensive code looks like:

```
if !v.IsValid()      { return errors.New("invalid value") }
if v.Kind() != reflect.Struct { return errors.New("not a struct") }
if !v.CanSet()       { return errors.New("not settable") }
```

It's verbose. Wrap in a helper if you do this a lot.

Interface() and unexported fields

`v.Interface()` panics if `v` refers to an unexported field. `v.CanInterface()` tells you ahead of time.

Nil checks

`reflect.Value.IsNil()` panics on kinds that aren't nillable. Use `Kind()` first or `CanInterface()` then a type switch.

```
switch v.Kind() {
case reflect.Pointer, reflect.Interface, reflect.Slice, reflect.Map, reflect.Chan,
     reflect.Func:
    if v.IsNil() { return }
}
```

Setting via map values

Map values are not addressable. You can't get one out, modify it, and have the map update.

```
m := map[string]Point{"a": {1, 2}}
v := reflect.ValueOf(m).MapIndex(reflect.ValueOf("a"))
v.FieldByName("X").SetInt(99)    // panic: not addressable
```

Workaround: pull out, modify a copy, set back.

```
p := m["a"]; p.X = 99; m["a"] = p
```

Or use `map[K]*V` so the values are pointers (which can be modified through reflection on what they point to).

Embedded fields

Promoted fields don't show up in `NumField()`. Walk the embedded field's fields recursively, or use `FieldByName` (which does follow promotions).

```
type Animal struct{ Name string }
type Dog struct{ Animal; Breed string }

t := reflect.TypeOf(Dog{})
t.NumField()           // 2 (Animal, Breed)
t.Field(0).Anonymous   // true (it's the embedded Animal)
f, _ := t.FieldByName("Name") // found via promotion
```

Type identity through reflection

```
reflect.TypeOf(int8(1)) == reflect.TypeOf(int8(2)) // true
reflect.TypeOf(MyInt(1)) == reflect.TypeOf(int(1)) // false (named type)
```

Two values have the same `reflect.Type` if and only if they have the same Go type.

Best practices

1. **Use reflection only when you can't know the type at compile time.** Almost everywhere else, generics, interfaces, or type switches are clearer and faster.
 2. **Cache type information.** Don't re-discover the same struct's fields on every call.
 3. **Guard every access with `Kind()` and `IsValid()` / `CanSet()` checks.** Otherwise you'll panic in production.
 4. **Wrap reflection behind a clean API.** Library callers shouldn't see `reflect.Value` types unless they need to.
 5. **Prefer codegen** for repetitive marshaling. `easyjson`, `stringer`, `mockgen`.
 6. **Document tag conventions** when you build a library that reads tags.
 7. **Use `(*T)(nil)` trick** to get types without values.
 8. **`reflect.DeepEqual` is for tests and small data.** Don't rely on it in hot paths.
 9. **Benchmark.** Reflection cost is real; measure before assuming the design works.
 10. **Read the standard library implementations.** `encoding/json`, `database/sql`, `text/template` all use reflection well; reading them is the best way to learn the idioms.
-

Common gotchas

Gotcha	Symptom	Fix
<code>Set</code> on non-addressable value	Panic	Start with a pointer, call <code>.Elem()</code>
<code>Set</code> on unexported field	Panic	Use exported field, or accept the limit
<code>Interface()</code> on unexported field	Panic	Check <code>CanInterface()</code>
<code>IsNil</code> on non-nillable kind	Panic	Check <code>Kind()</code> first
<code>Int()</code> on a string kind	Panic	Always check <code>Kind()</code>
Modifying map value through reflection	Panic	Read out, modify copy, write back
Forgetting <code>Elem()</code> after <code>reflect.New</code>	Wrong value type	<code>reflect.New(t).Elem()</code> for the value, <code>.Interface()</code> for <code>*T</code>
Comparing two <code>reflect.Value</code> directly	Doesn't compare contents	Use <code>Interface()</code> or <code>DeepEqual</code>
Reflection on a method that doesn't exist	<code>IsValid()</code> returns false; calling panics	Check <code>IsValid()</code>
Caching <code>reflect.Type</code> at package init	Works fine, but allocation visible	Acceptable
Re-walking the same type every call	Slow	Cache discovered structure
Calling exported method by name that takes <code>ctx</code>	<code>Call</code> argument mismatch	Wrap args correctly in <code>[]reflect.Value</code>
Reading tags with custom format	<code>Tag.Get</code> returns raw value	Parse format inside your code
<code>DeepEqual</code> on <code>time.Time</code> with monotonic clock	False inequality	Strip mono with <code>.Truncate(0)</code> first
Trying to compare functions	Always not equal unless both nil	Don't expect function equality

Gotcha	Symptom	Fix
Forgetting <code>reflect.Indirect</code> for pointer types	Empty inspection	Indirect follows pointers

Quick reference

What	Code
Get type	<code>reflect.TypeOf(v)</code>
Get value	<code>reflect.ValueOf(v)</code>
Kind	<code>t.Kind()</code> , <code>v.Kind()</code>
Name	<code>t.Name()</code>
Number of fields	<code>t.NumField()</code>
Field by index	<code>t.Field(i)</code> , <code>v.Field(i)</code>
Field by name	<code>t.FieldByName("X")</code> , <code>v.FieldByName("X")</code>
Tag	<code>field.Tag.Get("json")</code>
Is value valid	<code>v.IsValid()</code>
Is nil (nillable kinds)	<code>v.IsNil()</code>
Is zero	<code>v.IsZero()</code>
Can be set	<code>v.CanSet()</code>
Can be interfaced	<code>v.CanInterface()</code>
Dereference pointer	<code>v.Elem()</code> (also for interface)
Set int	<code>v.SetInt(x)</code>
Set string	<code>v.SetString(x)</code>
Get int	<code>v.Int()</code>
Convert to any	<code>v.Interface()</code>
Create *T zero	<code>reflect.New(t)</code>
Create slice	<code>reflect.MakeSlice(t, len, cap)</code>
Create map	<code>reflect.MakeMap(t)</code>
Create chan	<code>reflect.MakeChan(t, buf)</code>
Type from interface	<code>reflect.TypeOf((*MyIface)(nil)).Elem()</code>

What	Code
Implements check	<code>t.Implements(iface)</code>
Assignable check	<code>t.AssignableTo(other)</code>
Convertible check	<code>t.ConvertibleTo(other)</code>
Method by name	<code>v.MethodByName("X")</code>
Call function	<code>v.Call([]reflect.Value{...})</code>
Deep compare	<code>reflect.DeepEqual(a, b)</code>
Iterate map	<code>iter := v.MapRange(); for iter.Next() { ... }</code>
Indirect (follow pointer)	<code>reflect.Indirect(v)</code>
Zero value of type	<code>reflect.Zero(t)</code>